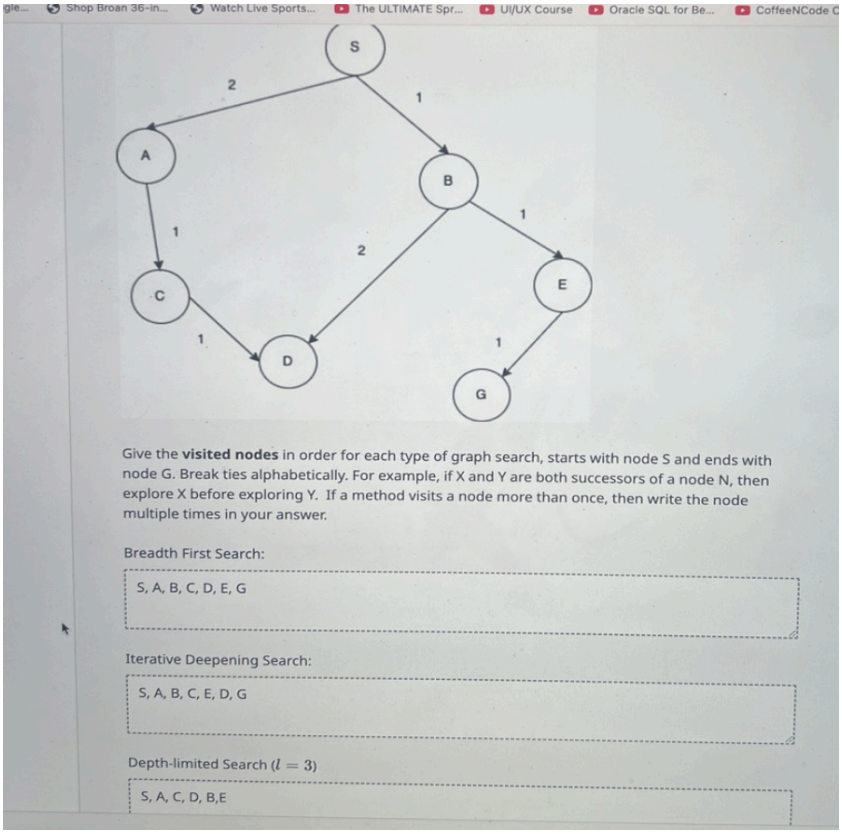
First, read the graph correctly

The directed edges are:

- S→AS \to AS→A
- S→BS \to BS→B
- A→CA \to CA→C
- C→DC \to DC→D
- B→DB \to DB→D
- B→EB \to EB→E
- E→GE \to GE→G

The edge costs matter for some searches, but for the parts visible in your screenshot, we mainly care about the **structure** of the graph and the rule:

- **Break ties alphabetically**
- So from SSS, explore **A before B**
- From BBB, explore **D before E**



Also, this says **graph search**, so once a node has already been visited in that same search run, we do **not** visit it again.

1) Breadth-First Search

Your answer shown is:

S, A, B, C, D, E, G

That is **correct**.

Why?

Breadth-first search explores **level by level**.

Think of BFS as using a **queue**:

- visit the start
- then all nodes 1 step away
- then all nodes 2 steps away
- then all nodes 3 steps away

Step-by-step

Start at:

- Visit **S**

Its children are:

- **A**, **B**
  Alphabetically: **A before B**

Queue now gives this visitation order next:

- **A**
- **B**

From **A**:

- child is **C**

From **B**:

- children are **D**, **E**
  Alphabetically: **D before E**

Then from **E**:

- child is **G**

So the BFS visit order is:

S, A, B, C, D, E, G.

2) Iterative Deepening Search

Key idea of IDS

Iterative deepening does:

- depth limit 0
- then depth limit 1
- then depth limit 2
- then depth limit 3
- and so on

It is basically **depth-first search repeated multiple times**, each time with a bigger depth limit.

Because the problem says:

> If a method visits a node more than once, then write the node multiple times

you must include the repeated visits across the different iterations.

Iteration 0 (limit = 0)

Only the start node can be visited:

S

Iteration 1 (limit = 1)

Start again from scratch:

- **S**
- children: **A**, **B**

So this iteration gives:

S, A, B

Iteration 2 (limit = 2)

Start again from scratch:

- **S**
- go depth-first to **A**
- then **C**
- backtrack
- then **B**
- then **D**
- then **E**

So this iteration gives:

S, A, C, B, D, E

Why is it **D before E** from BBB?
Because ties are broken alphabetically.

Iteration 3 (limit = 3)

Start again from scratch:

- **S**
- **A**
- **C**
- **D**
- backtrack to explore the other branch
- **B**
- from BBB: **D** and **E**
    - but this is **graph search**, so **D** was already visited in this iteration
    - so skip **D**
- go to **E**
- then **G**

So this iteration gives:

S, A, C, D, B, E, G

And we stop here because we found GGG.

Full IDS answer

Now concatenate all iterations:

- limit 0: **S**
- limit 1: **S, A, B**
- limit 2: **S, A, C, B, D, E**
- limit 3: **S, A, C, D, B, E, G**

Correct IDS order:

S, S, A, B, S, A, C, B, D, E, S, A, C, D, B, E, G

Why students often miss this

The most common mistake is writing only the last DFS-looking path.
But IDS is **multiple searches**, not one search.

3) Depth-Limited Search (l=3)(l = 3)(l=3)

The correct answer is:

S, A, C, D, B, E, G

Why?

Depth-limited search is just DFS, but you are not allowed to go deeper than the limit.

If the root SSS is depth 0, then:

- AAA, BBB are depth 1
- CCC, DDD, EEE are depth 2
- the lower DDD from CCC and GGG from EEE can be depth 3 depending on the path

The goal path is:

- S→B→E→GS \to B \to E \to GS→B→E→G

That is depth:

- SSS: 0
- BBB: 1
- EEE: 2
- GGG: 3

So **GGG is allowed**, because the limit is 3.

Step-by-step DFS order

Start at **S**.

Children of SSS:

- **A**, then **B**

Go depth-first left first:

- **S**
- **A**
- **C**
- **D**

Now backtrack to SSS, then go to BBB:

- **B**

Children of BBB:

- **D**, **E**

But this is graph search, and **D** was already visited, so skip it.

Then:

- **E**
- **G**

So the full depth-limited order is:

S, A, C, D, B, E, G

First, read the graph correctly

The directed weighted edges are:

- S→AS \to AS→A with cost 1
- S→GS \to GS→G with cost 12
- A→BA \to BA→B with cost 3
- A→CA \to CA→C with cost 1
- B→DB \to DB→D with cost 3
- C→DC \to DC→D with cost 1
- C→GC \to GC→G with cost 2
- D→GD \to GD→G with cost 3

Heuristic table:

h1h_1h1

- h1(S)=5h_1(S)=5h1(S)=5
- h1(A)=3h_1(A)=3h1(A)=3
- h1(B)=6h_1(B)=6h1(B)=6
- h1(C)=2h_1(C)=2h1(C)=2
- h1(D)=3h_1(D)=3h1(D)=3
- h1(G)=0h_1(G)=0h1(G)=0

h2h_2h2

- h2(S)=4h_2(S)=4h2(S)=4

- h2(A)=2h_2(A)=2h2(A)=2
- h2(B)=6h_2(B)=6h2(B)=6
- h2(C)=1h_2(C)=1h2(C)=1
- h2(D)=3h_2(D)=3h2(D)=3
- h2(G)=0h_2(G)=0h2(G)=0

Also, the problem says:

- break ties alphabetically
- path format is like S−A−BS-A-BS−A−B

Q1. What path would depth-first graph search return?

Correct answer:

S-A-B-D-G

Depth-first search goes as deep as possible before backing up.

Start at S.
Successors are:

- AAA
- GGG

Alphabetically, pick A first.

From A, successors are:

- BBB
- CCC

Alphabetically, pick B first.

From B, only successor is:

- DDD

From D, only successor is:

- GGG

So DFS reaches the goal along:

S-A-B-D-G

So the search path is:

- S→AS \to AS→A
- A→BA \to BA→B
- B→DB \to DB→D
- D→GD \to GD→G

That gives:

S-A-B-D-G

Q2. What path would A* graph search, using a consistent heuristic, return?

Correct answer:

S-A-C-G

Why?

A* uses:

f(n)=g(n)+h(n)

where:

- g(n) = actual cost from start to nnn
- h(n) = estimated cost from nnn to goal
- f(n)) = total estimated path cost

When the heuristic is consistent, A* graph search returns the optimal path.

So the easiest way to think about this question is:

> Find the cheapest path from S to G.

Compute the main path costs

Path 1:

S→G \to GS→G

Cost:

12

Path 2:

S→A→B→D→GS \to A \to B \to D \to GS→A→B→D→G

Cost:

1+3+3+3=101+3+3+3=101+3+3+3=10

Path 3:

S→A→C→D→GS \to A \to C \to D \to GS→A→C→D→G

Cost:

1+1+1+3=6 =

Path 4:

S→A→C→G S \to A \to C \to GS→A→C→G

Cost:

1+1+2=4

This is the cheapest path.

So A* with a consistent heuristic should return:

S-A-C-G

Intuition

DFS in Q1 just dove down the first branch.
A* is smarter: it uses costs and the heuristic to steer toward the cheapest-looking route.

Even though DFS found S−A−B−D−GS-A-B-D-GS−A−B−D−G, A* finds the cheaper route:

S-A-C-G

Q3. Is h1h_1h1 consistent? Is h1h_1h1 admissible?

Correct answer:

- h1h_1h1 is not consistent
- h1h_1h1 is not admissible

Part A: Is h1h_1h1 consistent?

Definition of consistency

A heuristic is consistent if for every edge

h(n)≤c(n,n′)+h(n′) That means:

> your estimated distance from the current node should not be more than
> "step cost to a neighbor + that neighbor's estimate"

Think of consistency as a triangle inequality for heuristics.

Check h1h_1h1 on the edge S→A S \to AS→A

We need:

h1(S)≤c(S,A)+h1(A)h_1(S) \le c(S,A)+h_1(A)h1(S)≤c(S,A)+h1(A)

Plug in values:

5≤1+3 This is false.

So h1h_1h1 is not consistent.

That alone is enough.

Part B: Is h1h_1h1 admissible?

Definition of admissible

A heuristic is admissible if it never overestimates the true cheapest cost to the goal.

So we first find the true cheapest cost from each node to G.

True cheapest costs to goal

From GGG:

000

From DDD:

D→G=3D \to G = 3D→G=3

From CCC:

Two choices:

- C→G=2C \to G = 2C→G=2
- C→D→G=1+3=4C \to D \to G = 1+3=4C→D→G=1+3=4

Cheapest:

2

From BBB:

Only useful route:

B→D→G=3+3=6B \to D \to G = 3+3=6B→D→G=3+3=6

From AAA:

Choices:

- A→C→G=1+2=3A \to C \to G = 1+2=3A→C→G=1+2=3
- A→C→D→G=1+1+3=5A \to C \to D \to G = 1+1+3=5A→C→D→G=1+1+3=5
- A→B→D→G=3+3+3=9A \to B \to D \to G = 3+3+3=9A→B→D→G=3+3+3=9

Cheapest:

3

From SSS:

Choices:

- S→G=12S \to G = 12S→G=12
- S→A→C→G=1+1+2=4S \to A \to C \to G = 1+1+2=4S→A→C→G=1+1+2=4

Cheapest:

Compare h1h_1h1 with true costs

- h1(S)=5h_1(S)=5h1(S)=5, true cost is 4 → overestimates
- h1(A)=3h_1(A)=3h1(A)=3, true cost is 3
- h1(B)=6h_1(B)=6h1(B)=6, true cost is 6
- h1(C)=2h_1(C)=2h1(C)=2, true cost is 2
- h1(D)=3h_1(D)=3h1(D)=3, true cost is 3
- h1(G)=0h_1(G)=0h1(G)=0, true cost is 0

Because h1(S)=5h_1(S)=5h1(S)=5 is bigger than the true cost 4, h1h_1h1 overestimates.

So h1h_1h1 is not admissible.

Best exam-style explanation for Q3

You could write:

> h1h_1h1 is not consistent because it violates the consistency condition on edge S→AS \to AS→A:
> h1(S)=5h_1(S)=5h1(S)=5 but c(S,A)+h1(A)=1+3=4c(S,A)+h_1(A)=1+3=4c(S,A)+h1(A)=1+3=4, and 5⩽45 \nleq 45⩽4.
> h1h_1h1 is also not admissible because it overestimates the true cheapest cost from SSS to GGG: the optimal cost is 4 via S−A−C−GS-A-C-GS−A−C−G, but h1(S)=5h_1(S)=5h1(S)=5.

Q4. Is h2h_2h2 consistent? Is h2h_2h2 admissible?

Correct answer:

- h2h_2h2 is not consistent
- h2h_2h2 is admissible

Part A: Is h2h_2h2 consistent?

Again check:

h(n)≤c(n,n′)+h(n′)h(n) \le c(n,n')+h(n')h(n)≤c(n,n′)+h(n′)

Look at edge S→AS \to AS→A:

h2(S)≤c(S,A)+h2(A)h_2(S) \le c(S,A)+h_2(A)h2(S)≤c(S,A)+h2(A)

Plug in:

4≤1+24 \le 1+24≤1+2 4≤34 \le 34≤3

False.

So h2h_2h2 is not consistent.

That's enough to conclude inconsistency.

Part B: Is h2h_2h2 admissible?

Compare each heuristic value to the true cheapest cost to goal.

True costs again:

- S=4S=4S=4
- A=3A=3A=3
- B=6B=6B=6
- C=2C=2C=2
- D=3D=3D=3
- G=0G=0G=0

Now compare:

- h2(S)=4≤4h_2(S)=4 \le 4h2(S)=4≤4
- h2(A)=2≤3h_2(A)=2 \le 3h2(A)=2≤3
- h2(B)=6≤6h_2(B)=6 \le 6h2(B)=6≤6
- h2(C)=1≤2h_2(C)=1 \le 2h2(C)=1≤2
- h2(D)=3≤3h_2(D)=3 \le 3h2(D)=3≤3
- h2(G)=0≤0h_2(G)=0 \le 0h2(G)=0≤0

It never overestimates.

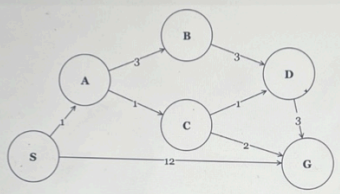
So h2h_2h2 is admissible.

Best exam-style explanation for Q4

You could write:

> h2h_2h2 is not consistent because it violates consistency on edge S→AS \to AS→A: h2(S)=4h_2(S)=4h2(S)=4, while c(S,A)+h2(A)=1+2=3c(S,A)+h_2(A)=1+2=3c(S,A)+h2(A)=1+2=3, so 4⩽34 \nleq 34⩽3.
> h2h_2h2 is admissible because for every node, the heuristic value is less than or equal to the true cheapest cost to the goal. For example, the true costs are S=4,A=3,B=6,C=2,D=3,G=0S=4, A=3, B=6, C=2, D=3, G=0S=4,A=3,B=6,C=2,D=3,G=0, and h2h_2h2 never exceeds these.

Given the graph and heuristic table shown below, answer the following questions. Break ties alphabetically.
(S stands for start, G stands for goal. A path from S to B is represented as S-A-B.)



State	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Q1 What path would depth-first graph search return for this search problem?

S, A, B, D, G

Q2 What path would A* graph search, using a consistent heuristic, return for this search problem?

S, A, C, G

Q3 Is h_1 consistent? Is h_1 admissible? Explain how you know to receive credit.

h1 is consistent as the cost of each h1 path is less than the adjacent node cost actual cost to the goal node plus it's current heuristic value. h1 is admissible as all the heuristic values of the nodes do not overestimate the actual distances (less than) to the goal node.

Q4 Is h_2 consistent? Is h_2 admissible? Explain how you know to receive credit.

H2 is not consistent as the cost of each h2 path is more than the adjacent node cost actual cost to the goal node plus it's current heuristic value. h2 is admissible as not all the heuristic values of the nodes do not overestimate the actual distances (less than) to the goal node. Some do overesitmate.